Quantum Tensor-Network Contraction Optimization

Parallel Systems(CSC548) Course Project

Srikar Chundury
North Carolina State University
Raleigh, USA
schundu3@ncsu.edu

Abstract

This paper presents an effort to parallelize a previously sequential numerical library, which is being developed as part of the author's thesis project, with the goal of enabling the simulation of 2D tensor network-based quantum circuit simulations. The library, written in C++, exploits a sparsity pattern observed in the state vector simulation of quantum circuits, and employs multi-threading via OpenMP for format conversion and the SIMD vectorization technique for a portion of the contraction operation. Tests are conducted using chain matrix multiplication of Qiskit unitaries and quimb's 2D tensor networks to evaluate the performance of the parallelized library in comparison to the sequential version. The results indicate that OpenMP significantly improves format conversion(∼**64.9%** for 26 qubit GHZ unitary matrix), while SIMD vectorization has little impact on overall run-time performance. Also, we discovered a sizeable performance improvement when transitioned from using std::vector to regular arrays in our library optimization efforts (∼**13.7%** for 26 qubit GHZ chain matrix multiplication). This ongoing work suggests that the library will continue to evolve and improve as it is further optimized for quantum circuit simulations, and it has the potential to make significant contributions to the field of quantum computing.

Keywords: Quantum Computing, High Performance Computing, Tensor Networks, Parallel Systems

1 Introduction

Quantum computing holds immense potential to transform fields like cryptography [22], [2], optimization [10] [19], physics/chemistry [15], [26], machine learning [4], [12], and finance [28], [5]. Despite its promise, simulating large quantum circuits is a daunting task due to the exponential growth of the state space [29]. Tensor networks have emerged as a useful tool to overcome this challenge [27]. Tensors are multi-dimensional maps or functions, with a 2-dimensional tensor equivalent to a matrix. Tensor networks are modeled as graphs, with tensors as vertices and tensor methods, like contraction and decomposition, as edges. They have broad implications in various fields, including applied mathematics, chemistry, physics, and machine learning [3].

There are different types of tensor networks based on their dimensionality and adjacency [24]. One-dimensional tensor networks, also known as Matrix Product States (MPS) or Tensor Trains (TT), and higher-dimensional tensor networks, like Projected Entangled Pair States (PEPS), have mainly been employed as a proposed quantum wavefunction for the ground states of two-dimensional Hamiltonians [15], [18]. A pictorial depiction of MPS and PEPS can be seen in Figures 4 and 1, respectively.

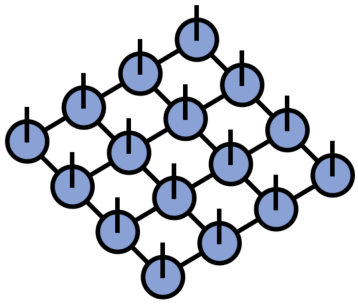


Figure 1. A visual representation of the Projected Entangled Pair States (PEPS) tensor network.

Efficient tensor-network based quantum simulation is of paramount importance due to the vast potential applications of quantum computing and the usefulness of tensor networks in simulating quantum circuits. Quantum hardware is expensive, and preparing quantum states takes a considerable amount of time, making it impractical to run multiple attempts while developing new quantum algorithms. Moreover, current quantum computers are prone to errors due to problems such as decoherence and noise, which can limit the accuracy of quantum algorithms [20]. On the other hand, classical simulation of quantum circuits is memory-intensive, and with every additional qubit, the memory requirements double, making efficient contraction algorithms that can "evolve" through the tensor network a crucial factor for achieving better performance. Therefore, developing efficient tensor network-based quantum simulation methods can have numerous applications, including simulating complex quantum systems for research purposes and optimizing quantum algorithms, and can help overcome these challenges in quantum computing.

In simulating quantum circuits using tensor networks, there are several general steps involved. Firstly, the tensor network representation of the circuit is built by adding tensors representing the gates to the initial state, which is usually the product state $|000...00\rangle$. Low-rank decompositions may also be performed on the tensors if necessary. Secondly, the entire tensor network of the desired quantity is formed, which could be the full wavefunction, a local expectation value or reduced density matrix, a marginal probability distribution, or the fidelity with a target state or unitary [11].

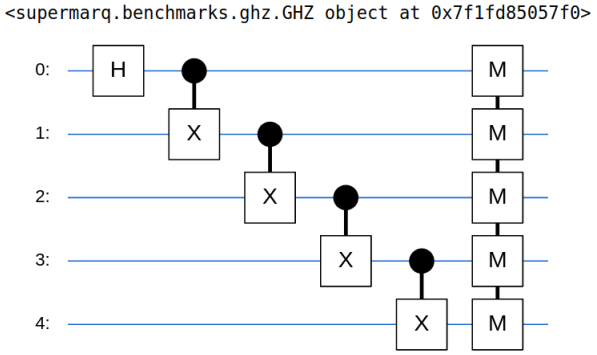


Figure 2. SupermarQ GHZ quantum circuit.

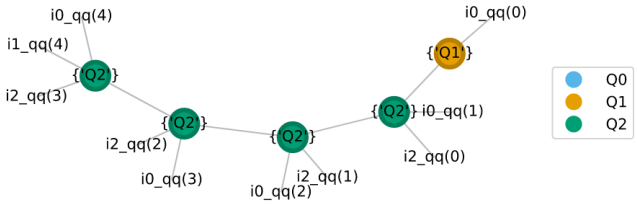


Figure 3. SupermarQ GHZ Tensor-Network Representation. The circles represent tensors in the network, for instance, the yellow circle represents the Hadamard gate and all the green ones represent CNOT gates. The lines (edges) represent bond interactions. The measurements have been removed, since this is a unitary simulation.

As an example, the Figure 2 shows a quantum circuit and Figure 3 shows its tensor network representation.

Local simplifications are then performed to make the tensor network easier to contract, followed by finding a contraction path to turn the tensor network into a single tensor. The contraction may also be sliced or chunked to fit memory constraints or introduce parallelism. Finally, the resulting tensor network can be contracted using

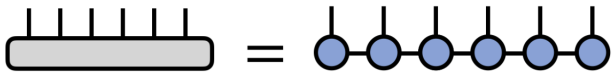


Figure 4. A visual representation of the Matrix Product State (MPS) tensor network.

specialized libraries, such as our numerical library, to efficiently perform the necessary calculations.

In the context of the author's ongoing thesis project, a novel data structure has been proposed which aims to represent the specific sparsity pattern observed in quantum unitaries concisely, leading to a reduction in memory consumption and computation time during matrix operations. To this end, this course project is dedicated to investigating the possibility of parallelizing certain components of this library in order to enhance their performance. The primary objectives of this work are to enable the parallelization of the format conversion process from the standard dense matrix representation to the newly proposed data structure, and to vectorize a portion of the spM-spM kernel. It is worth noting that while various sparse matrix formats such as COO, CSR, and CSC are available, we claim that this proposed data structure is best suited to meet the demands of quantum simulation, as will be demonstrated in the course of the thesis project. However, it is out of the scope of this project to delve into the specifics of the proposed data structure and its performance analysis. Instead, this project solely focuses on exploring potential parallelizations in the library to further improve certain kernels.

As a baseline to compare performances of various numerical libraries, we chose the SupermarQ benchmark suite [25]. SupermarQ is a comprehensive benchmark suite designed by Infleqtion [13], a part of ColdQuanta Labs [8], for evaluating the performance of quantum computers. It consists of eight different benchmarks that cover a broad range of quantum applications, including QAOA [10], VQE [19], GHZ [9], Hamiltonian [15], Mermin-Bell [6], error correction, phase-code, and bit-code. The principles of design behind SupermarQ include scalability, meaningfulness, full stack evaluation, adaptability, hardware agnostic program profiles, connectivity, and parallelism. The benchmark suite allows for the evaluation of a variety of architectures, as well as the observation of the impact of mid-circuit measurement, using T1/readout metrics. We conduct experiments involving the simulation of three circuits from this benchmark suite, namely GHZ, Hamiltonian (or ham), and Mermin Bellpair generation (or mermin_bell).

The remainder of the term paper is outlined as follows- we begin with a study of similar tools and libraries in section x. In section x we describe our approach and discuss why we do not pursue certain other approaches. We then step thorough the experiments conducted in section x and discuss the results in section x. Finally, we close with the conclusion and future scope remarks in section x.

2 Literature Review

This section presents a comparative analysis of our high-performance numerical library for quantum computing with other existing quantum libraries and their supported numerical libraries.

2.1 Cyclops - CTF/Koala

Cyclops Tensor Framework (CTF) [23] is a high-performance numerical library for multidimensional arrays (tensors) in C++ and Python designed for parallel computing. The library utilizes MPI communication to coordinate tensor operations across all processes executing the program. Cyclops supports near sparsity using the COO (coordinate list) format, various symmetries, and user-defined element types, making it a versatile tool for scientific applications such as physics, chemistry, engineering, and machine learning. Its support for near sparsity allows it to efficiently handle large-scale data sets with complex data structures, while its support for symmetries can significantly reduce computational costs for certain operations.

Cyclops is interoperable with other scientific computing libraries such as ScaLAPACK and numpy, allowing for seamless integration into existing workflows and the ability to leverage its capabilities alongside other tools. These features make Cyclops an efficient and versatile option for researchers and scientists working with large-scale data sets and complex tensor operations.

Koala [17] is a quantum circuit simulation library that is based on projected entangled-pair states (PEPS) tensor networks. It is also developed by the same group. It currently supports numpy and ctf as tensor backends.

Like CTF, the author's thesis project also involves the development of a novel high-performance numerical library for quantum computing. This library is aimed at presenting sparsity through the utilization of innovative data structures and specialized kernels. Although the library is still in progress, it is intended to be integrated with Koala, which supports PEPS tensor networks. At present, the implementation of the library at a library-level is yet to be explored and is therefore considered out of scope for this course project.

2.2 Qiskit - Numpy/Jax

Qiskit [21] is a high-performance quantum simulation library that provides users with a streamlined workflow consisting of four high-level steps: Build, Compile, Run, and Analyze. Users design a quantum circuit(s) to represent their problem, compile it for a specific quantum service, and then run the compiled circuits on a specified quantum service(s) that can be cloud-based or local. Finally, users analyze the results by computing summary statistics and visualizing the experiments. This workflow enables users to efficiently perform quantum simulations and computations using Qiskit. As per qiskit textbook, the only available backends for qiskit are numpy and jax. Both of which only have support for dense arrays.

Our work is primarily different from these libraries with respect to the way they are stored, utilizing the seen sparsity pattern and hence reducing memory requirements.

In our project, we utilize Qiskit library to read and collect intermediate unitaries. These unitaries are then arranged in sequence, using chain matrix multiplications, to measure specific metrics that will be discussed in detail in the methodology section.

2.3 Quimb - opt_einsum

The Quimb [11] library offers strong support for the simulation of quantum circuits, leveraging its capacity to depict and collapse arbitrary geometry tensor networks. However, its depiction typically falls outside the category of representing the complete wavefunction (like many other simulators) or a specific type of TN (such as an MPS or PEPS, as with some other simulators), which necessitates a unique approach to utilizing it that may require additional consideration. Google's Cirq also uses this package internally for it's tensor-network based simulation needs. The opt_einsum package [1] has an option to switch between various numerical libraries like torch, jax, numpy, tensorflow and more, for the contraction step.

In this work, we leverage this package to compare performance of tensor-network simulation of supermarQ benchmark circuits for these libraries against our new library.

3 Methodology

In this section, we will discuss the parallelization approaches used in the numerical library for optimizing the performance of the library. Specifically, we compare the dense kernel with the sparse kernel for a sequence of matrix multiplications representing a quantum circuit simulation using the C++ library. We observe in Figure 5 that most of the time is spent in the conversion step

from the dense format to our format. This observation is also confirmed by linux perf hotspots.

To tackle this performance bottleneck, we use OpenMP [7] to implement multi-threading on the format conversion step. This parallelization approach enables us to utilize multiple CPU cores for converting dense matrices to our format, with the intent of reducing the conversion time.

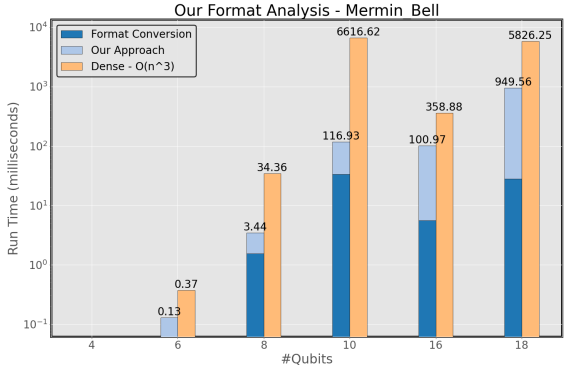


Figure 5. Dense Kernel vs Sparse Kernel for SupermarQ's mermin_bell benchmark. Note that 16 and 18 qubit simulations are based on generated data.

In addition to the format conversion step, we also parallelize two other sections of the library - reshape operation and transpose operation. The parallelization of these operations allows us to utilize multiple CPU cores for performing these operations, thereby further improving the overall performance of the library.

Within the sparse matrix times sparse matrix kernel, we make use of SSE4.2 for SIMD vectorization [14] . This approach enables us to perform multiple calculations simultaneously using a single instruction, thereby improving the performance of the sparse matrix times sparse matrix multiplication operation.

3.1 OMP Parallelization

To convert a matrix from the dense format to our format, we iterate through every element in the dense matrix, which has n^2 elements, and perform a constant time operation on each element.

We also use OMP in the reshape and transpose operations which are important operations used by the einsum operation.

3.2 SIMD Vectorization

In quantum simulations, we often deal with complex numbers. However, SSE4.2 SIMD instructions do not directly support complex numbers. To address this issue, we split an array of complex numbers into two arrays - a real number array and a complex number array. This approach allows us to use two SSE4.2 '___m128' vectors to perform operations on the arrays.

To perform complex vector times complex vector, we use two real vectors and two imaginary vectors. We multiply each element of the real vector with the corresponding element of the other real vector and each element of the imaginary vector with the corresponding element of the other imaginary vector. We then subtract the product of the imaginary vectors from the product of the real vectors to get the real part of the result. Similarly, we add the product of the real and imaginary vectors and multiply it by -1 to get the imaginary part of the result.

After performing the complex vector operation, we stitch the resulting real and imaginary vectors back into a complex vector again. We take care of the left-over values (values that are not pure multiples of 4) by using partial loads.

We later modified this version to avoid the split and stitch which we talk about in the improvements subsection.

Overall, this approach enables us to perform operations on complex numbers using SSE4.2 SIMD instructions, with the intent of improving the performance of the numerical library for quantum simulations.

3.3 Hybrid Version

Due to the fact that the aforementioned two sections optimize distinct independent parts of the library, we intend to integrate them cohesively in order to achieve an overall enhancement in performance of the library.

3.4 Improvements

In our investigations of performance optimizations, we observed that SIMD operations over complex number arrays incurred certain performance overheads. Specifically, we noted the necessity of splitting complex number arrays into their real and imaginary components prior to vectorization. To mitigate this issue, we chose to store the real and imaginary components of complex numbers separately from the outset. This modification resulted in some improvement in performance.

In addition, we opted to shift away from using dynamic vectors (i.e., the std::vector data structure of C++11) in favor of regular arrays. Notably, this alteration directly enhanced the performance of the OpenMP version. For instance, the runtime for a 24 qubit GHZ circuit chain multiplication with std::vector was measured to be approximately 0.63745 seconds, while the runtime with regular arrays was measured to be approximately 0.00185 seconds, representing a percent reduction in time of 99.71%

We attribute this improvement to the internal dynamic behavior and implementation of std::vector.

#Qubits	Operations	Numerical Libraries compared	#Iterations
4,6,8,10,12,16	Tensor-Network Simulation, Chain Matrix Multiplication	Numpy, Jax, Tensorflow, Scipy (COO Format), Our Library	50
(Generated data) 16,18,20,22,24,26	Chain Matrix Multiplication, Reshape Operation, Transpose Operation	Dense O(n^3) Version, Our Library	50

Table 1. Summary of all the experiments performed.

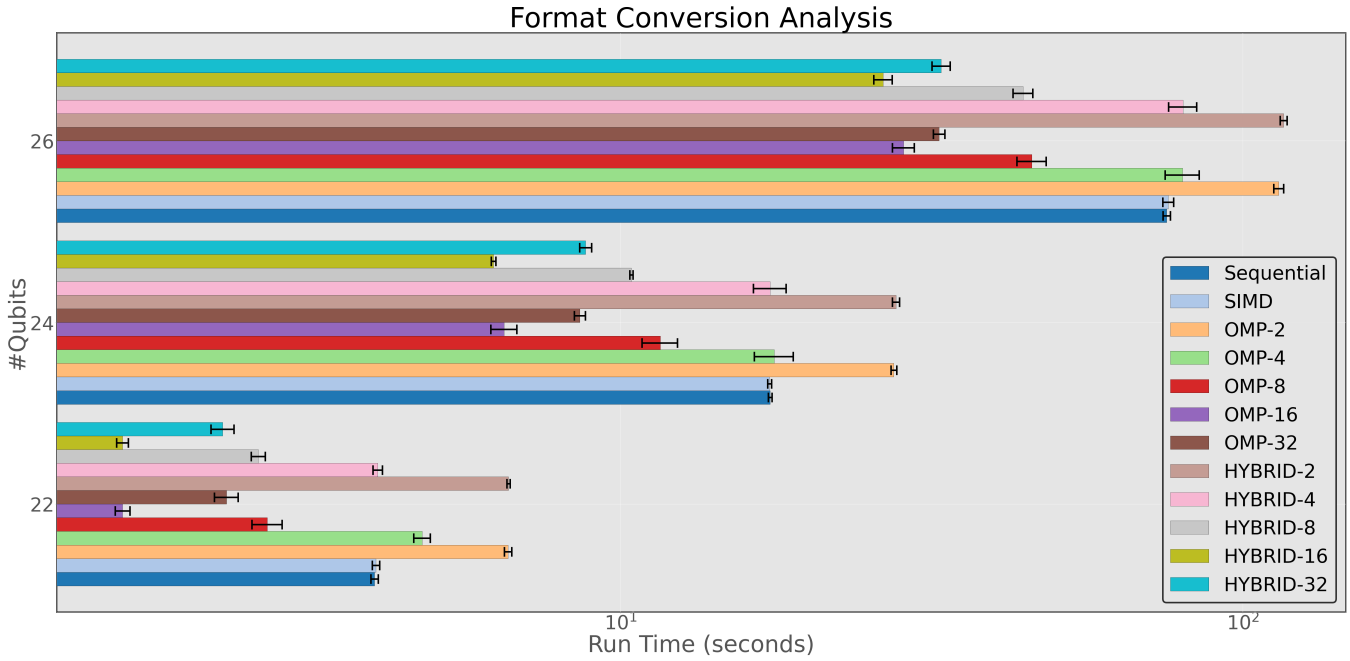


Figure 6. Comparison of various versions that run the format conversion operation.

4 Experiments

This section outlines the tests conducted on the different versions of our C++ library. The supermarQ benchmark circuits were employed for two extensive experiments. Firstly, we utilized Qiskit's unitary simulator to gather intermediate unitary matrices, which were subsequently subjected to chain multiplications using our C++ library for the three versions. Secondly, our C++ library was integrated with the Quimb library, which facilitated the conversion of a given quantum circuit to its tensor network equivalent. This enabled us to simulate the circuit using the opt_einsum package with different backends, including our library, for the three versions.

Since, collecting intermediate matrices for wider quantum circuits is exponentially complex interms of memory requirements, we scale up the relatively smaller intermediates to generate similar data for larger number of qubits (>=16). For instance, we stitch four 8-qubit matrices to form a 16-qubit unitary.

The experiments are summarized in Table 1.

All tests have been run on a single node of Intel's Cascade (2.50 GHz) processor. This node has 192GB of DDR4 RAM and 32 logical cores. It has 64KB of L1 cache, 1MB of L2 cache, and 10MB of L3 cache. This is part of the ARC cluster [16] in the CSC department at NC State University.

5 Results & Discussion

We present the results obtained using the new version of the library after replacing std::vector with arrays, for format conversion and matrix multplication. Our optimized implementation using OpenMP with 16 threads in parallel outperforms all other implementations, with a 65% improvement in format conversion for a 26 qubit circuit, as shown in Figure 6.

We further analyze the performance of modern numerical libraries for tensor network-based simulations using opt_einsum, a package used internally by Quimb. Our

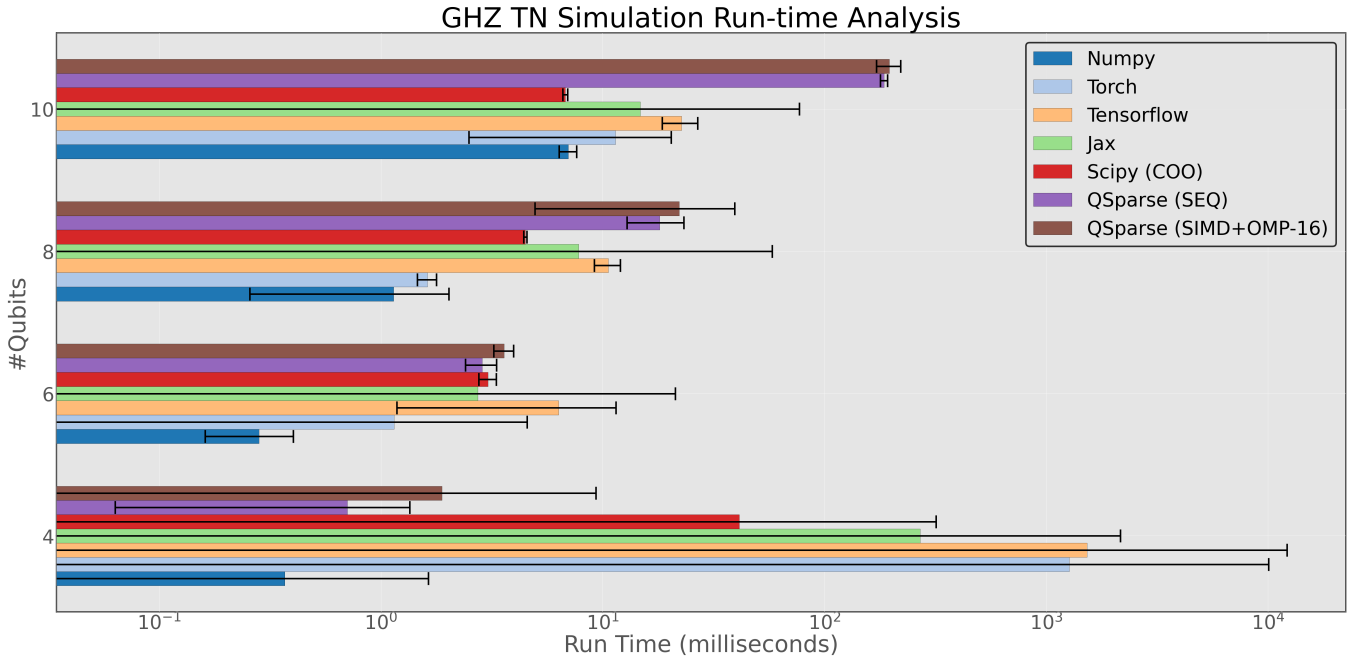


Figure 7. Performance comparison of modern numerical libraries for tensor network-based simulations using opt_einsum.

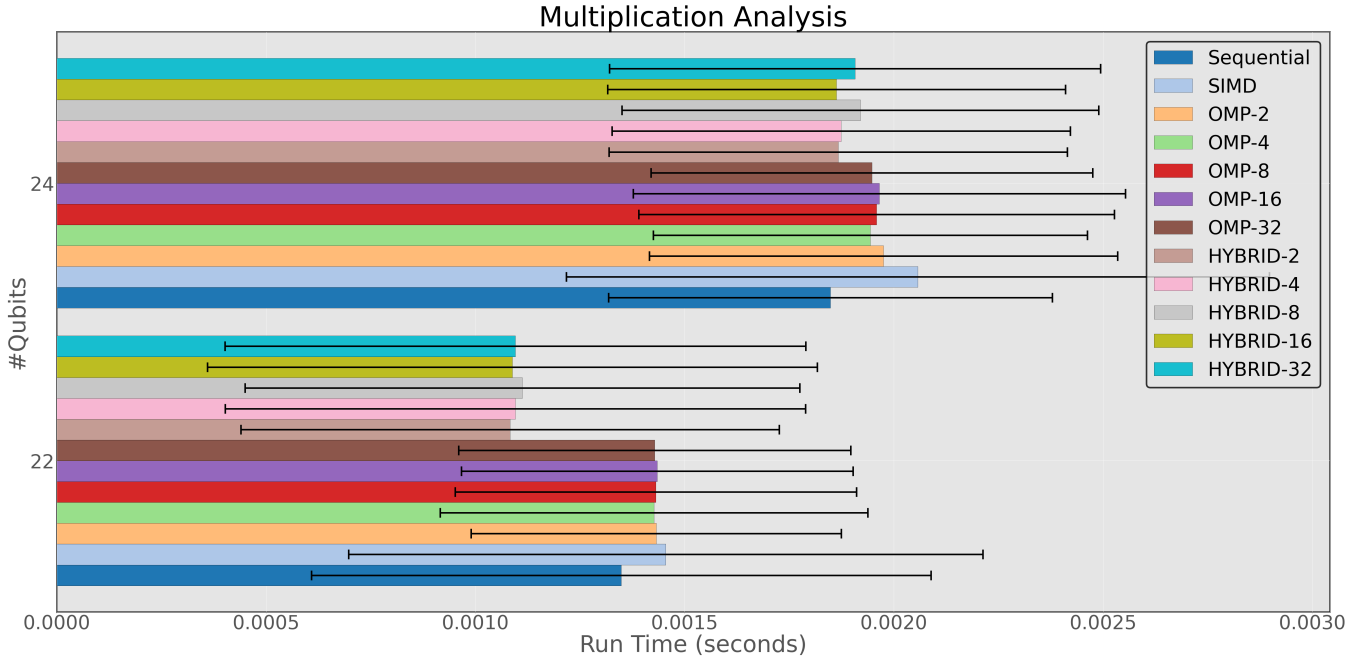


Figure 8. Multiplication performance analysis of various versions on large GHZ intermediate matrices.

implementation provides an interface to link to different libraries, and we compare its performance with other libraries in Figure 7.

We also examine the impact of SIMD on performance in Figure 8, where we observe that for diagonals of size 2^{26}, the run-time is low (in the order of microseconds) and we cannot definitively conclude that the sequential version is always better. The error plots also support this conclusion.

The novelty of our approach lies in its ability to simulate >20 qubit circuits using our format, where other packages such as "sparse" throw "too many dimensions passed to ravel_multi_index," and "numpy" throws "numpy.core._exceptions.MemoryError" due to an inability to store such large matrices in memory.

6 Conclusion

In this study, we aimed to optimize the performance of a previously sequential numerical library for the simulation of 2D tensor network-based quantum circuit simulations. To achieve this goal, we employed parallelization approaches, including multi-threading via OpenMP and SIMD vectorization using SSE4.2, to improve the performance of the library. Specifically, we focused on optimizing the format conversion step, reshape operation, transpose operation, and sparse matrix times sparse matrix kernel.

We conducted tests using chain matrix multiplication of Qiskit unitaries and quimb's 2D tensor networks to evaluate the performance of the parallelized library in comparison to the sequential version. Our results showed that OpenMP significantly improves format conversion, while SIMD vectorization has little impact on overall run-time performance. Additionally, we discovered a substantial performance improvement when transitioning from using std::vector to regular arrays in our library optimization efforts.

Furthermore, we made some improvements to the implementation of the library by splitting complex number arrays into their real and imaginary components prior to vectorization, and using regular arrays instead of dynamic vectors, resulting in significant improvements in performance.

Overall, our optimized implementation and format conversion approach show promising results for large-scale quantum circuit simulations.

7 Future Work

Firstly, it is necessary to compare the library's implementation with that of open BLAS 3's cGEMM implementation. Secondly, it is imperative to explore better utilization of cache through techniques like blocking. It is anticipated that utilizing the library will enable simulation of very large qubit circuits (>20) that cause other libraries like numpy to run out of memory.

However, Quimb's end-to-end contraction process with our library is not as efficient as with numpy, necessitating further investigation, particularly in optimizing reshape and transpose operations. In addition, exploring the possibility of developing a quantum algorithm level library like Koala/Qiskit/Quimb that is tailored to the numerical library and stores gates in the library's format would eliminate the need for any format conversions and is expected to result in the greatest performance gains.

Furthermore, we plan to explore the possibility of identifying sub-circuits that can be efficiently simulated using the data structure through techniques like memoization.

Acknowledgement

I would like to extend my sincere gratitude to Dr. Jiajia Li and Dr. Frank Mueller for their invaluable guidance and inputs throughout my Parallel Systems course project. Their consistent feedback and support have been instrumental in helping me navigate the complexities of this project and make significant progress towards the right direction. I am truly grateful for their mentorship and expertise, which have greatly enriched my learning experience.

Appendix

- The sequential code is available at https://github.ncsu.edu/tnqs/sparse-tnqs/tree/srikar-dev.
- The OMP code version is available at https://github.ncsu.edu/tnqs/sparse-tnqs/tree/srikar-dev-omp.
- The SIMD code version is available at https://github.ncsu.edu/tnqs/sparse-tnqs/tree/srikar-dev-simd.
- The SIMD+OMP code version is available at https://github.ncsu.edu/tnqs/sparse-tnqs/tree/srikar-dev-hybrid.
- The older code, i.e. the code with std::vector is available at https://github.ncsu.edu/tnqs/sparse-tnqs/tree/old-srikar-dev; other branches- old-srikar-dev-omp, old-srikar-dev-hybrid and old-srikar-dev-simd.
- All raw files and relevant jupyter notebooks with these results are available in the code repository https://github.ncsu.edu/tnqs/benchmarks under directories bench and quimb.

References

[1] Daniel G. a. Smith and Johnnie Gray. opt_einsum - a python package for optimizing contraction order for einsum-like expressions. *Journal of Open Source Software*, 3(26):753, 2018.

[2] Eric R. Anschuetz, Jonathan P. Olson, Alán Aspuru-Guzik, and Yudong Cao. Variational quantum factoring, 2018.

[3] Jacob Biamonte and Ville Bergholm. Tensor networks in a nutshell, 2017.

[4] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, sep 2017.

[5] Lee Braine, Daniel J. Egger, Jennifer Glick, and Stefan Woerner. Quantum algorithms for mixed binary optimization applied to transaction settlement. *IEEE Transactions on Quantum Engineering*, 2:1–8, 2021.

[6] José L. Cereceda. Mermin's n-particle bell inequality and operators' noncommutativity. *Physics Letters A*, 286(6):376–382, aug 2001.

[7] Rohit Chandra, Leo Dagum, David Kohr, Ramesh Menon, Dror Maydan, and Jeff McDonald. *Parallel programming in*

OpenMP. Morgan kaufmann, 2001.
[8] cql. Cold quanta labs, 2023.
[9] Manuel Erhard, Mehul Malik, Mario Krenn, and Anton Zeilinger. Experimental greenberger–horne–zeilinger entanglement beyond qubits. *Nature Photonics*, 12(12):759–764, oct 2018.
[10] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm, 2014.
[11] Johnnie Gray. quimb: A python package for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.
[12] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical Review Letters*, 103(15), oct 2009.
[13] infq. Infleqtion website, 2023.
[14] lol. Simd sse4.2, 2023.
[15] Guang Hao Low and Isaac L. Chuang. Hamiltonian simulation by qubitization. *Quantum*, 3:163, jul 2019.
[16] notimp. Arc cluster @ ncsu, 2023.
[17] Yuchen Pang, Tianyi Hao, Annika Dugad, Yiqing Zhou, and Edgar Solomonik. Efficient 2d tensor network simulation of quantum systems. pages 1–14, 11 2020.
[18] David Perez-Garcia, Frank Verstraete, J. Ignacio Cirac, and Michael M. Wolf. Peps as unique ground states of local hamiltonians, 2007.
[19] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5(1), jul 2014.
[20] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, aug 2018.
[21] qiskit. Qiskit simulators, 2023.
[22] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, oct 1997.
[23] Edgar Solomonik, Devin Matthews, Jeff R Hammond, John F Stanton, and James Demmel. A massively parallel tensor contraction framework for coupled-cluster computations. *Journal of Parallel and Distributed Computing*, 74(12):3176–3190, 2014.
[24] tensornetworks. Tensor networks, 2023.
[25] Teague Tomesh, Pranav Gokhale, Victory Omole, Gokul Subramanian Ravi, Kaitlin N. Smith, Joshua Viszlai, Xin-Chuan Wu, Nikos Hardavellas, Margaret R. Martonosi, and Frederic T. Chong. Supermarq: A scalable quantum benchmark suite, 2022.
[26] A. V. Uvarov, A. S. Kardashin, and J. D. Biamonte. Machine learning phase transitions with a quantum processor. *Physical Review A*, 102(1), jul 2020.
[27] Harm van Eersel. Tensor network methods for quantum simulation. 2011.
[28] Stefan Woerner and Daniel J. Egger. Quantum risk analysis. *npj Quantum Information*, 5(1), feb 2019.
[29] Alwin Zulehner and Robert Wille. Advanced simulation of quantum computations, 2018.